Repetition Statements

while-, for-, do while statements

Review

- Variable name
  - Start with a letter or _
  - Followed by letters, digits, or _
  - No keywords of C++ is allowed
- A variable needs to be declared exactly once before usage
  - Usage can be initialization – set values use assignment or input from console
- branch statements: if-, if-else, nested if-else

A story of if statement

Ann: **If** you have $1000, will you please give me a half?
Bob: Of course.

Ann: **If** you have $100, will you please give me a half?
Bob: Sure.

Ann: **If** you have $10, will you please give me a half?
Bob: No way!!
Ann: Why???
Bob: I do not have $100 or more, but I do have $10.

if statement

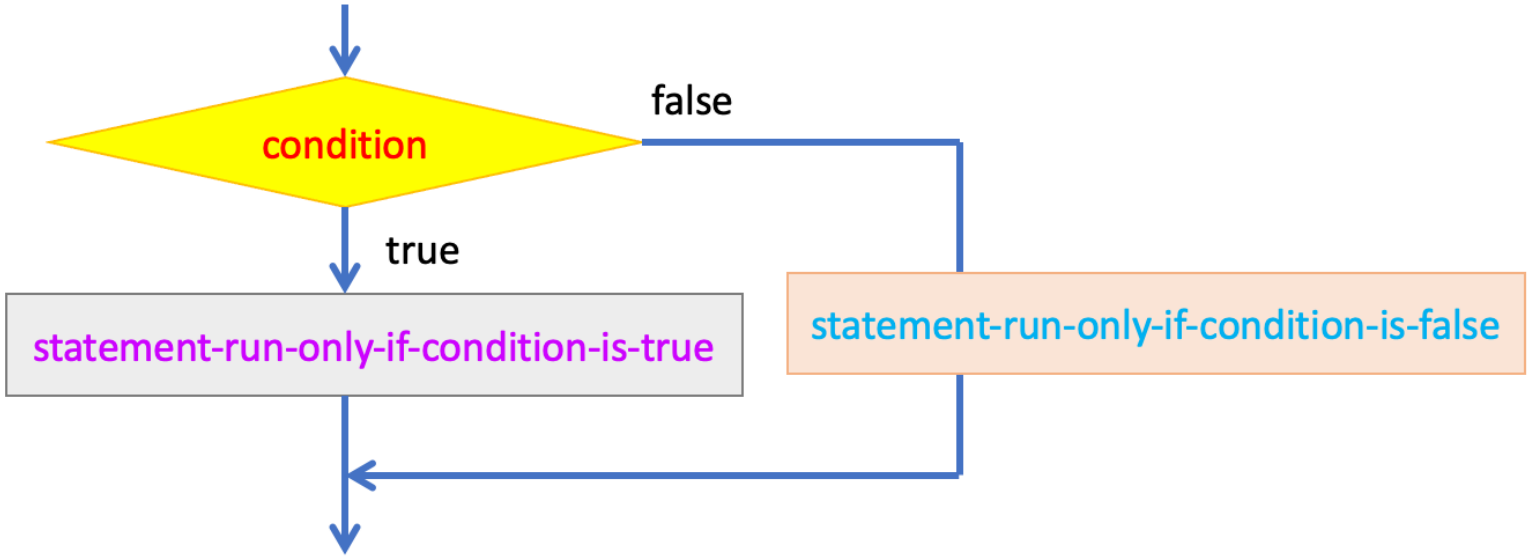
- Only run statements in if-body when conditions are satisfied. Do nothing when condition is not satisfied.
- Example: To find out the cutest baby model, do the following:
  - Let the first baby be the cutest baby model – no competitor yet.
  - If a new baby is cuter than the current cutest baby model, update the current cutest baby model by the new baby, otherwise, do nothing – keep the current status quo.

If statement: II

- Similar examples:
  - If the new comer is stronger than the current strongest, update the current strongest to the new comer, otherwise, nothing is needed.
  - If the new comer is weaker than the current weakest, update the current weakest to be the new comer, otherwise, nothing is needed.
- Sometimes conditions need to be combined by && (and) or || (or).
  - If you graduate from a NY high school, or your GPA is 3 out of 4, then you can apply for finance aids.

If-else statement

- When conditions hold, do something, otherwise, do other thing.
- Like a T stop, take either if- or else- branch.



Common mistakes

- No () to enclose condition.
- Add ; right after the condition.
- What is difference between the following pseudocode?

```
if (the sun rises from west)
    I will give you a million;
```

```
if (the sun rises from west) ;
    I will give you a million;
```

=

```
if (the sun rises from west)
    ;
I will give you a million;
```

It takes two operands to compare

Suppose variable color is properly declared and initialized. Which is correct condition to represent that color is not ”red”, “green”, or “yellow”?

(color != ”red”, “green”)

(color != “red” ||
color != ”green”)

(color != “red” &&
color != ”green”)

courtesy https://grammarist.com/idiom/it-takes-two-to-tango/

not a word, and not a sound

Leap Year

- In the **modern Gregorian calendar**, a year is a *leap year* if it is divisible by 4, but century years are not leap years unless they are divisible by 400.
- That is, a leap year is one of the following
  - Divisible by 4 but not by 100

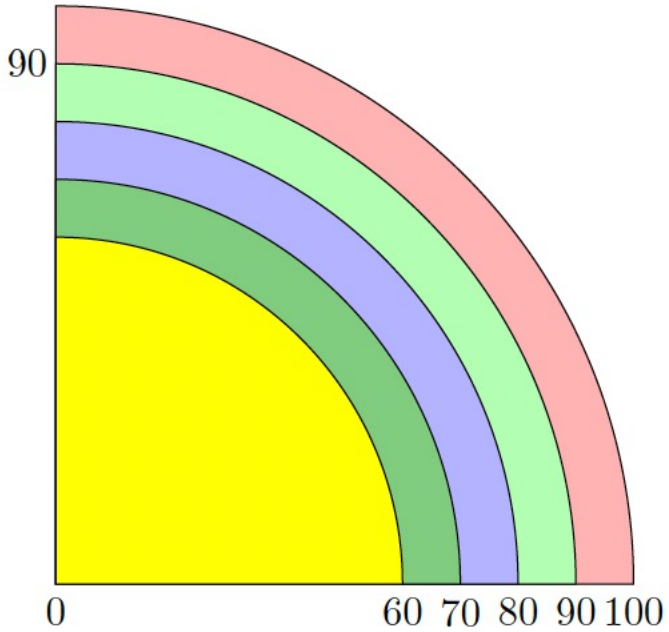
  OR
  - Divisible by 400

Nested if-else: Categorize mutually exclusive cases

- A traffic light is exactly one of the following: red, green, or yellow.
- A number is exactly one of the following: positive, zero, or negative.
- A letter grade is exactly A, B, C, D, or F.

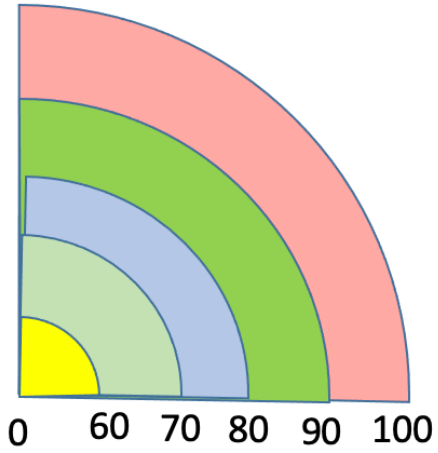
Convert Numerical Grade to Letter Grade

- If grader is >= 90, print ‘A’.
- If grade is in [80, 90), print ‘B’, and so on.



Order of nested if-else statement is important

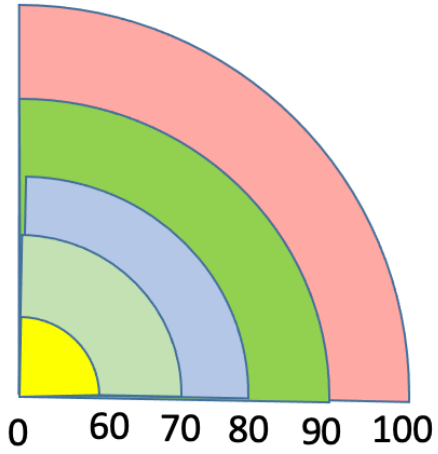
```
if (score >= 90)
    cout << "A" << endl;
else …
```



Peel from outermost layer, then move inward.

Order of nested if-else statement is important

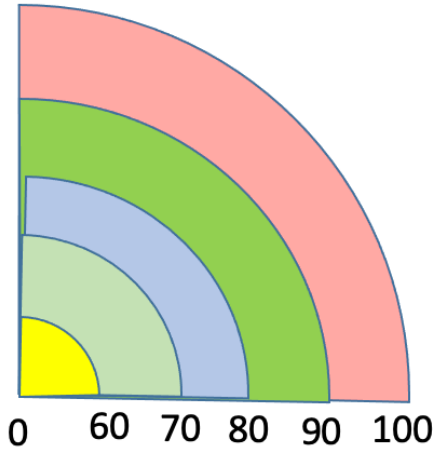
```
if (score >= 90)
   cout << "A" << endl;
else if (score >= 80)
        cout << "B" << endl;
else if (score >= 70)
            …
else if (score >= 60)
            …
else …
```



Peel from outermost layer, then move inward.

Order of nested if-else statement is important: II

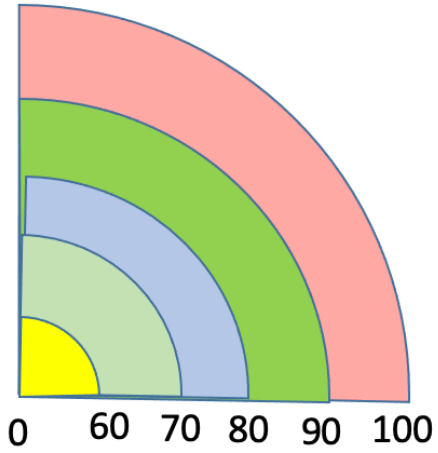
```
if (score < 60)
   cout << "F" << endl;
else if (score < 70)
        cout << "D" << endl;
else if (score < 80)
            …
else if (score < 90)
            …
else …
```



Peel from innermost layer, then move outward.

Order of nested if-else statement is important: III

```
if (score >= 70)
    // NO conclusion
    // on letter grade!
```



Can we peel from middle layer? Say 70?

Nested if-else is used for mutually exclusive cases

- Nested if-else is to categorize cases, which need to be mutually exclusive.

```
//Declare and initialize a, b, c. Code omitted. What if a, b, c are 1, 2, 3?
int maxVal = a;
if (b > maxVal) //stronger than the current strongest
   maxVal = b;
else if (c > maxVal)
         //wrong.  (b > maxVal) and (c > maxVal) not mutually exclusive
        maxVal = c;
```

Arithmetic operators +, -, *, /, %

- When both divisor and dividend are integers, the quotient is also integer. For example, 3 / 2 returns 1.

Calculate the following:

(3*2)/4

3*(2/4)

4/(2*3)

3/2.0

Repetition

- Keep on doing something until the condition is no longer satisfied
- Include while-, for-, and do-while statements

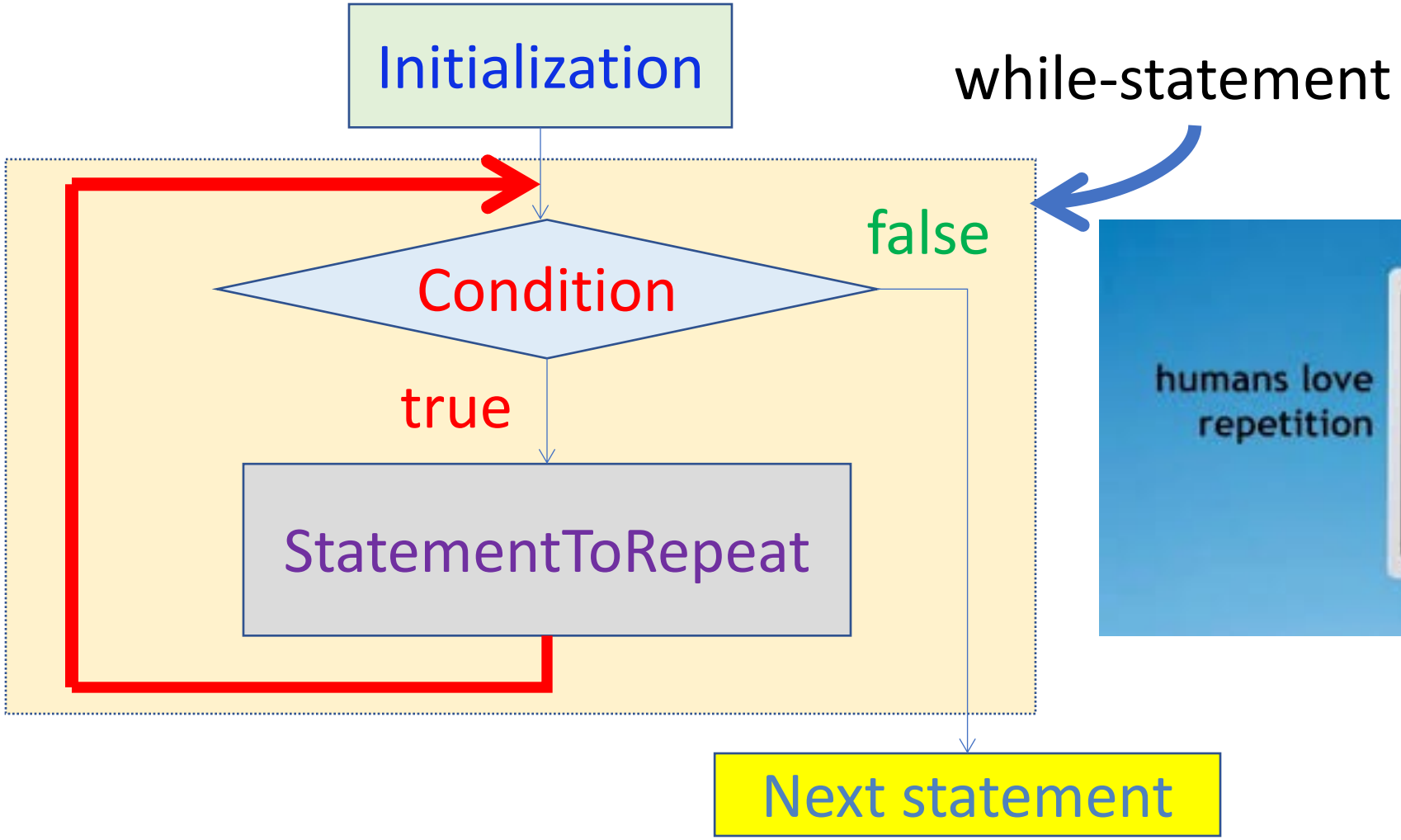
while-statement

Initialization of variable used in condition;

```
while (condition) //while-head
        statementToRepeat;  //while-body
```

- Do not forget the initialization part before while-header.
- As long as condition is satisfied, run statementToRepeat.
- If there are more than one statement, put them inside a pair of { and }.

Illustration of while-statement



Details for while-statement

- What is the minimum number the statementToRepeat will run?
- What is the maximum number the statementToRepeat will run?
  - What if the condition is always satisfied?
  - Common sense: the variable used in condition should be updated by statementToRepeat.

Keep on running until reaching a goal

In view of distance:

Where do we start?	Mile mark 1
Where is our goal?	Mile mark 5
How fast do we go in each step?	Jump 1 mile
What do we do in each step?	Leave a paw mark

Keep on working until reaching a goal

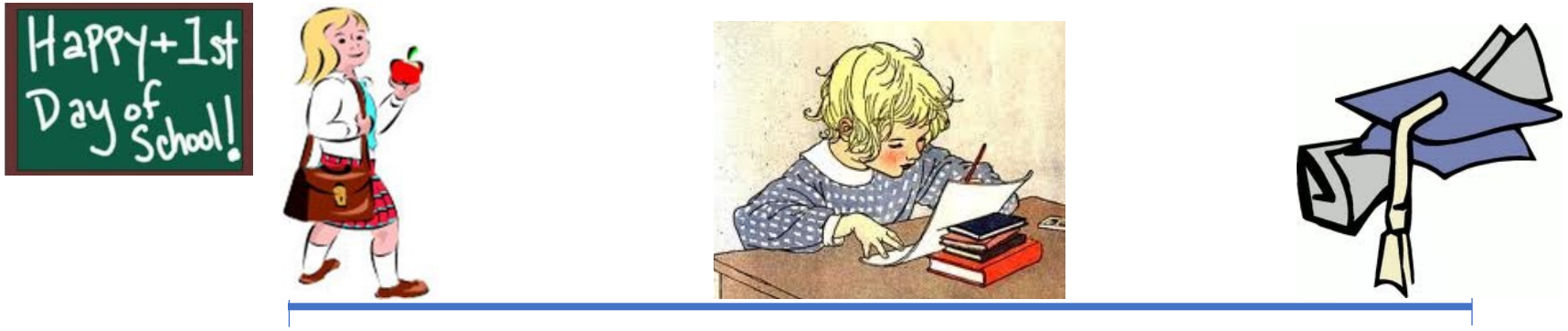
<u>In view of timeline:</u>

When do we start? Freshman or Transfer junior?

What is our goal? Associate or Bachelor degree?

How fast do we go in each step? Take 9 or 18 credits a semester?

What do we do in each step? Take necessary courses or play?



start goal

Repetition In term of time

Start at 1 o’clock

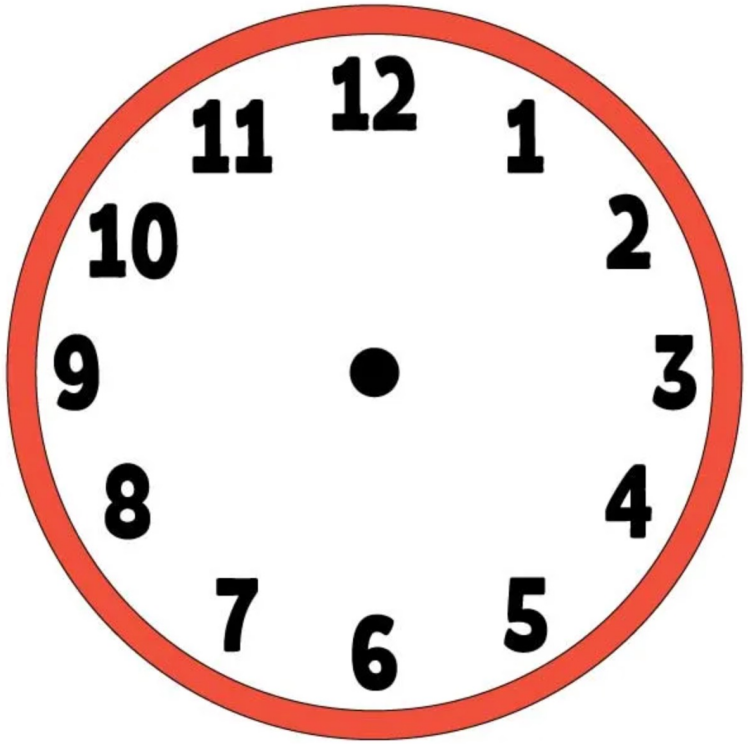
As long as not 5 o’clock yet

begin

Do something

Move to next o’clock

end



Philosophy of while-statement

1. Initialize the loop variable.
2. Check condition
3. if the loop condition is satisfied
   1) Use loop variable (depending on the problem)
   2) Update loop variable.
4. Go back to Step 2.

while-statement is stronger than if-statement.

Initialization of loop variable is important

```
#include <iostream>
using namespace std;

int main() {
  int num = 0;
  while (num < 0) {
    cout << "Enter num until it is non-negative: ";
    cin >> num;
  }
  return 0;
}
```

ENSURE to input a non-negative integer

```
int main() { //online gdb link
    cout << "Enter an non-negative int: ";
    int num;
    cin >> num; //initialization: enter num from console
    while (num < 0) {
        cout << "Enter num until it is non-negative: ";
        cin >> num; //update loop variable
    }
    return 0;
}
```

Modify the code to ensure input in [0, 100]

```
int main() {
    cout << "Enter a non-negative int: ";
    int num;
    cin >> num; //initialization: enter num from console
    while (num < 0) {
        cout << "Enter num until it is non-negative: ";
        cin >> num;
    }
    return 0;
}
```

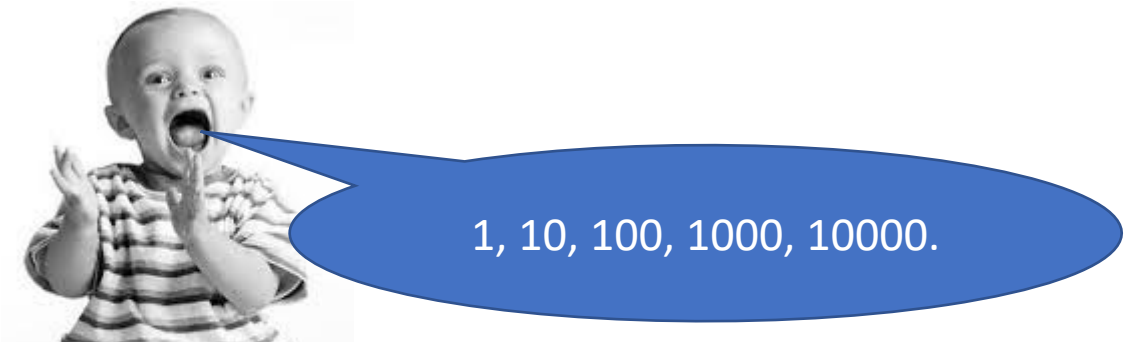
The following code cannot ensure the input in [0, 100]

```
#include <iostream>
using namespace std;

int main() {
    int num;
    cout << "Enter a number: ";
    cin >> num;

    if (num > 0 || num < 100) {
        cout << "nice";
    }
    else {
            cout << "enter a number: ";
            cin >> num;
    }
    return 0;
}
```

- (num > 0 || num < 100) should be (num >= 0 && num <= 100)
- What happens if we input -1, then -2?

A joke: count from 1 to 10000

- A kid is asked to count from 1 to 10000.



- Rational of the story: how to update variable is important.

Count from 1 to 10000 (both ends included)

```
int main() { //online gdb link
    int i = 1;
    while (i <= 10000) {
        cout << i << endl;
        i *= 10;
    }
    return 0;
}
```

Trace a loop

```
int i = 1;
while (i <= 10000) {
    cout << i << endl;
    i *= 10;
}
```

Loop variable	condition	Loop body	
i	i <= 10000	cout << i << endl	i *= 10
1	yes	Print 1, then new line	i changes to 10
10	yes	Print 10, then new line	i changes to 100
100	yes	Print 100, then new line	i changes to 1000
1000	yes	Print 1000, then new line	i changes to 10000
10000	yes	Print 10000, then new line	i changes to 100000
100000	NO		

Investment

Start from $10000, with annual interest rate 5%, how many years to get AT LEAST $20000?

Investment code

```
double amount = 10000; //online gdb link
double rate = 0.05; //cannot write 5%
int year = 0;
while (amount < 20000) {
    amount *= 1 + rate;
    year++;
    //cout << "year " << year << ": " //print intermediate results
    //     << amount << endl;
}
cout << "take " << year << " years "
     << "to get from 10000 to at least 20000" << endl;
cout << "amount = " << amount << endl;
```

Add all inputs until -1 is entered

```
Enter a number (-1 to stop):  5
Enter a number (-1 to stop):  6
Enter a number (-1 to stop):  -2
Enter a number (-1 to stop):  -1
sum is 9
```

Transfer the above request as

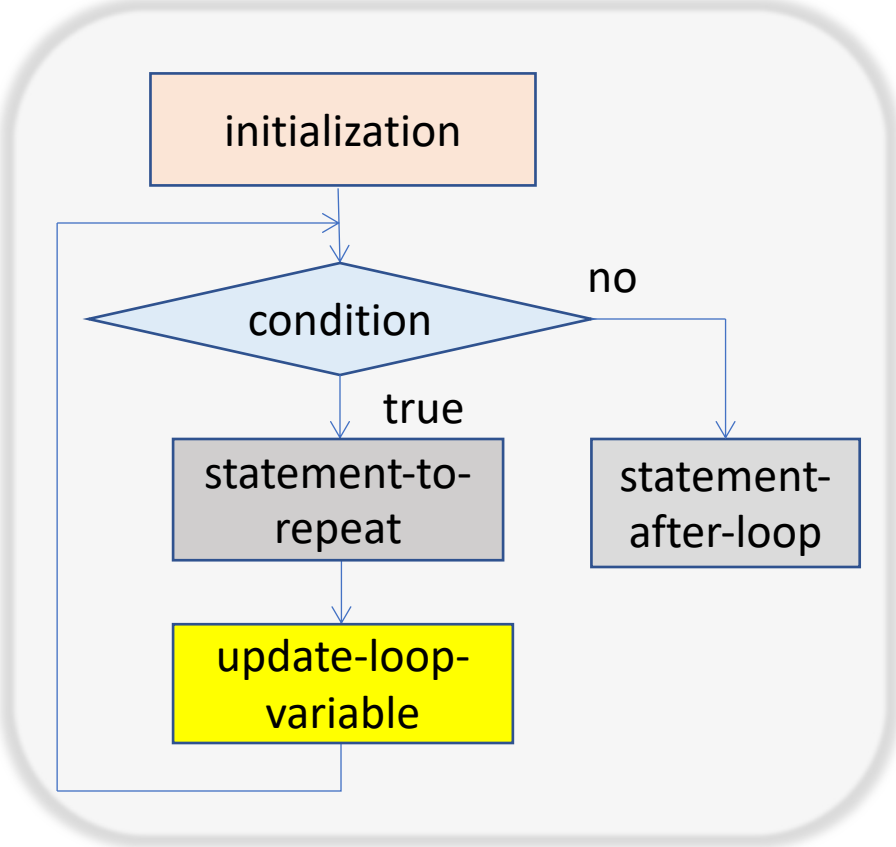
```
initialization
while (……) // while is equivalent to as long as
    do something
```

Code to add all inputs until -1 is entered

```
cout << "Enter a number (-1 to stop): "; //link
int num;
cin >> num;
int sum = 0; //initialize sum to be 0
while (num != -1) {
    sum += num;
    cout << "Enter a number (-1 to stop): ";
    cin >> num;
}
cout << "sum = " << sum << endl;
```

for-statement

```
                ❶                 ❷                        ❹
for (initialization; condition; update-loop-variable) {
    statement-to-repeat;
                ❸
}
statement-after-loop;
                ❺
```



for-statement

- Put initialization of a loop variable and update the loop variable after each loop inside the for header.

Do something for 100 times

```
int i = 0;
while ( i < 100 ) {
    do something;
    i++;
}
//can access variable i
cout << i << endl; //valid
```

```
int i; //declaration
for (i = 0; i < 100; i++) {
    do something;
}
//variable i is declared
//before for-statement,
//can be accessed outside
//the for-statement
cout << i << endl; //valid
```

Example: do something for 100 times: II

```
int i = 0;
while ( i < 100 ) {
        do something;
        i++;
}
//can access variable i
cout << i << endl; //valid
```

```
for (int i = 0; i < 100; i++) {
    do something;
}
//variable i is declared
//inside for-statement,
//cannot be accessed outside
//the for-statement
```
~~cout << i << endl;~~ //invalid

Print all integers from 1 to 100

- For statement is normally used when number of repetitions are known in advance.
- Where does the number start?
- Where does the number end?
- What do we do in each round?
- Do we update the number in each round?

Code to print all integers from 1 to 100

```
for (int i = 1; i <= 100; i++) { //link
    //no space between < and =

    cout << i << endl;
    //loop body has only one statement,
    //{} to enclose the loop body is optional,
    //but encouraged.
}
```

Add all integers in [1, 100]

- Where does the number start?
- Where does the number end?
- What do we do in each round?
- Do we update the number in each round?

Code to add all ints in [1, 100]

```
int main() { //online gdb link
    int sum = 0;
    for (int i = 1; i <= 100; i++){
        sum += i;
    }

    cout << "sum = " << sum << endl;
    return 0;
}
```

Take home message

- If you know to do something for n times, where n is an integer. Use for-statement.
- For example, print “No dumb questions” for 10 times. Print * for n times, where n is initialized.

```
for (int i = 0; i < n; i++){
  //n is declared and initialized before loop
  do something;
}
```

do-while statement

```
initialization;
do {
    statements;
} while (condition) ;
//Do not forget ; after the condition in do-while statement.
//This ; ends the do-while statement.
//Comparison:  do not add ; after condition in while-statement,
//or the loop body of while-statement is empty – nothing to repeat.
```

Difference between do-while and while-statement

```
initialization;
while (condition) {
        statements;
}
```

- Check at entry
- something needed to done zero, one, or many times

```
initialization;
do {
    statements;
} while (condition) ;
```

- Check at exit
- something needed to done at least once (can be many times)

Examples to use do-while statement

- Enter a number in [0, 100]. If it is not in the range, force the user to re-input until we get a number in [0, 100].
  - Need to enter a number for at least once.
- Find out number of digits of an integer.
  - Each integer at least one digit to extract.

Number of digits of an int: do-while statement

```
cout << "enter an int: "; //online gdb link
int num;
cin >> num;
int numDigits = 0;
do {
    num /= 10;
    numDigits++;
} while ( num != 0 );

cout << "# of digits: " << numDigits << endl;
```

Number of digits of an int: while-statement

```
cout << "enter an int: "; //online gdb link
int num;
cin >> num;

num /= 10; //same as num = num / 10;
int numDigits = 1; //extract one digit by num /= 10;
while (num != 0) {
    num /= 10;
    numDigits++;
}

cout << "# of digits: " << numDigits << endl;
```

Ensure an input is in [0, 100]

```
prompt to enter an int in [0, 100]
declare integer variable num
enter an integer to num
while (num is not in [0, 100])
{
    prompt to re-enter
    enter an integer to num
}
```

```
declare integer variable num
do
{
    prompt to enter an int in [0, 100]
    enter an integer to num
} while (num is not in [0, 100]);
```

while-, for-, and do-while statements

- If the number of repetitions is known in advance, use for-statement.
  - for-statement explicitly includes initialization and update-variable statement and is more succinct than while-statement.
- Normally, do-while statement can be rewritten as a while-statement, but not vice versa.
  - while statement may not run statements inside loop body, however, do-while statement needs to run statements inside loop body at least once, you may not be able to undo already-run-statement, like print.